

UNIVERSIDAD DE LA SABANA

Maestría — Módulo de Observabilidad

Laboratorio técnico — Pipeline de observabilidad con OpenTelemetry

Pipeline de Observabilidad End-to-End con OpenTelemetry

*Trazas, métricas y logs correlacionados por W3C Trace Context,
de Docker Compose a Google Cloud Run*

Integrantes del grupo:

Nestor Alejandro Rodriguez Benavides — nestorrobe@unisabana.edu.co

Carlos Daniel Sandoval — carlossandpar@unisabana.edu.co

Peter Alexander Palacios Garnica — peterpaga@unisabana.edu.co

Agosto de 2026

Repositorio: github.com/0DanielS/OpenTelemetry-End-to-End-con-Jaeger

Índice

1. Resumen ejecutivo
2. Visión general y arquitectura
 - 2.1 Arquitectura (local y nube)
 - 2.2 Flujo de una orden (request lifecycle)
3. Stack tecnológico
4. Instrumentación con OpenTelemetry
 - 4.1 Auto-instrumentación
 - 4.2 Spans custom y métricas SLI
 - 4.3 Logs estructurados
5. Verificación de la correlación cross-signal (W3C Trace Context)
6. Backends y visualización
 - 6.1 OTel Collector (local y nube)
 - 6.2 Dashboards Grafana
7. Análisis de overhead (k6, local)
8. Despliegue en GCP (Cloud Run + Cloud SQL)
 - 8.1 Arquitectura del despliegue
 - 8.2 CI/CD y seguridad (Workload Identity Federation)
 - 8.3 Telemetría nativa en GCP
9. Pruebas de carga en la nube y diagnóstico de incidente
10. Guía de ejecución local y despliegue
11. Decisiones de diseño y lecciones aprendidas
12. Conclusiones

1. Resumen ejecutivo

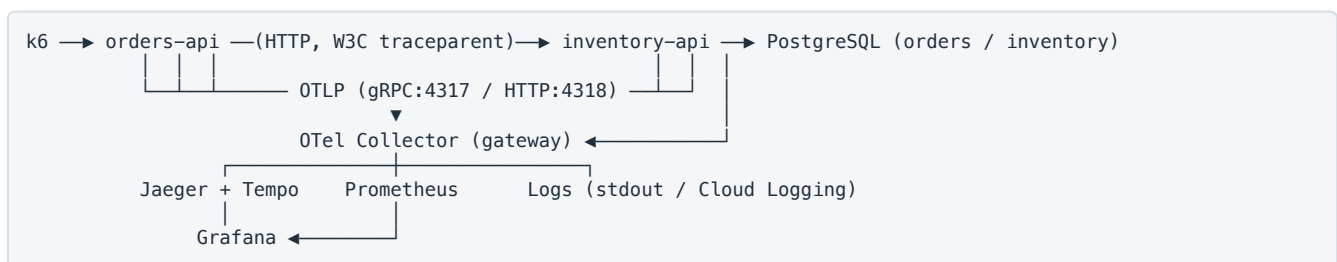
Se implementó y validó un **pipeline de observabilidad end-to-end** basado en OpenTelemetry sobre dos microservicios Python/FastAPI (`orders-api` → `inventory-api`), capturando las tres señales —**trazas, métricas y logs estructurados**— y correlacionándolas por `trace_id` según el estándar **W3C Trace Context**. Un OpenTelemetry Collector desacopla las aplicaciones de los backends: en local enruta a Jaeger, Tempo y Prometheus con visualización en Grafana; en la nube exporta a Cloud Trace, Cloud Logging y Managed Prometheus.

El despliegue productivo se realizó en **GCP Cloud Run + Cloud SQL PostgreSQL** con CI/CD por GitHub Actions autenticado mediante **Workload Identity Federation** (sin llaves de servicio) y secretos en Secret Manager, bajo guardrails de coste que apagan la base de datos fuera de sesión. El benchmark de overhead local cuantificó el costo de la instrumentación (−38% throughput, +31% p99, +19–27% memoria), y las pruebas de carga en la nube demostraron el valor operativo del pipeline: un agotamiento de conexiones en Cloud SQL fue **diagnosticado con la propia telemetría** (trazas, métricas y logs) y corregido hasta lograr una prueba con 100% de éxito.

2. Visión general y arquitectura

2.1 Arquitectura (local y nube)

La solución se compone de dos microservicios con dependencia HTTP, cada uno con su propia base de datos, y un plano de observabilidad desacoplado vía OTLP:



El mismo diseño corre en **dos entornos equivalentes**; solo cambian los backends:

Plano	Local (Docker Compose)	Nube (GCP)
Servicios	Contenedores (orders :8080, inventory :8081)	Cloud Run (escala a cero)
Base de datos	PostgreSQL 16 (contenedor)	Cloud SQL PostgreSQL 16 (<code>db-f1-micro</code>)
Trazas	Jaeger :16686 y Tempo :3200	Cloud Trace
Métricas	Prometheus :9090	Managed Prometheus (GMP)
Logs	stdout JSON (structlog)	Cloud Logging (filtrables por <code>trace_id</code>)
Dashboard	Grafana :3000 (10 paneles)	Grafana en Cloud Run (9 paneles)

2.2 Flujo de una orden (request lifecycle)

El cliente hace `POST /orders` → `orders-api` valida la entrada (Pydantic) → llama por HTTP a `inventory-api` (`POST /products/{id}/reserve`) propagando el header `traceparent` → `inventory-api` descuenta stock con un **UPDATE atómico** (nunca deja stock negativo bajo concurrencia) → `orders-api` inserta la orden y responde 201. Cada salto genera spans de auto-instrumentación más spans custom de negocio, métricas SLI y logs con el `trace_id` del recorrido.

3. Stack tecnológico

Capa	Tecnología	Rol
Servicios	Python 3.12 · FastAPI · uvicorn · httpx · SQLAlchemy async + asyncpg · Pydantic · structlog	Microservicios y acceso a datos
Instrumentación	OpenTelemetry SDK 1.29.0 · instrumentaciones 0.50b0 (<code>opentelemetry-instrument</code>)	Trazas, métricas y contexto W3C
Pipeline	OTel Collector contrib 0.109 (OTLP gRPC/HTTP)	Recepción, procesamiento y enrutado de señales
Backends locales	Jaeger 1.57 · Tempo 2.5 · Prometheus 2.54 · Grafana 11	Almacenamiento y visualización
Backends GCP	Cloud Trace · Cloud Logging · Managed Prometheus	Telemetría nativa administrada
Datos	PostgreSQL 16 (local) / Cloud SQL (nube)	Bases <code>orders</code> e <code>inventory</code>
Despliegue	Docker Compose (local) · Cloud Run + Cloud Build + Artifact Registry (nube)	Ejecución y empaquetado
CI/CD y seguridad	GitHub Actions · Workload Identity Federation · Secret Manager	Despliegue automático sin llaves
Carga	k6 (Grafana)	Benchmark de overhead y pruebas de saturación

4. Instrumentación con OpenTelemetry

4.1 Auto-instrumentación

Ambos servicios se ejecutan bajo `opentelemetry-instrument` (CLI de `opentelemetry-distro`), que aplica instrumentación automática de **FastAPI** (spans server), **httpx** (spans client con propagación W3C automática) y **SQLAlchemy** (spans connect/execute con `db.statement`, `db.system`). El `Resource` se enriquece vía `OTEL_RESOURCE_ATTRIBUTES` con `service.version`, `deployment.environment` y `cloud.provider`, atributos que viajan en cada señal.

4.2 Spans custom y métricas SLI

Señal	Elemento	Servicio	Propósito
Trazas	<code>persist_order</code>	orders-api	Span de negocio sobre el INSERT de la orden
Trazas	<code>check_stock</code> / <code>reserve_stock</code>	inventory-api	Spans sobre la consulta y el descuento atómico de stock
Métricas	<code>http_requests_total</code>	ambos	SLI de disponibilidad/errores (counter con label <code>status</code>)
Métricas	<code>http_request_duration_milliseconds</code>	ambos	SLI de latencia (histograma; percentiles p50/p95/p99)
Métricas	<code>http_active_requests</code>	ambos	SLI de saturación (requests en vuelo, up-down counter)
Métricas	<code>service_b_calls_total</code> / <code>inventory_requests_total</code>	orders / inventory	Métricas de negocio de las llamadas cross-service
Métricas	<code>db_operation_duration</code> (ms)	ambos	Latencia DB — necesario porque SQLAlchemy async no emite <code>db.client.operation.duration</code>

Las métricas SLI se emiten desde un **middleware propio** en cada servicio (los health checks se excluyen), y el pool de conexiones a la base de datos es **configurable por entorno** (`DB_POOL_SIZE` / `DB_MAX_OVERFLOW`: 20/40 en local, 5/5 en Cloud Run — ver sección 9).

4.3 Logs estructurados

Logs en JSON (structlog) con un processor que inyecta `trace_id` y `span_id` desde el span activo:

```
{"order_id": "da65151f-...", "event": "order.created", "trace_id": "3d6cf90ef96226881054ba92f6594942",  
"span_id": "be787879b17fac07", "level": "info", "timestamp": "2026-08-17T02:07:05.350200Z"}
```

5. Verificación de la correlación cross-signal (W3C Trace Context)

El requisito crítico es que el `trace_id` sea consistente entre las tres señales. Se validó en local con un trace real (`3d6cf90ef96226881054ba92f6594942`):

- **Logs:** el JSON de `order.created` contiene el mismo `trace_id` (sección 4.3).
- **Trazas:** Jaeger muestra el árbol completo con 15 spans; los spans de `inventory-api` son hijos del span `POST /orders` de `orders-api`, confirmando la propagación `traceparent` vía httpx.
- **Métricas:** etiquetadas por `service_name` gracias a `resource_to_telemetry_conversion: enabled` en el exporter Prometheus.

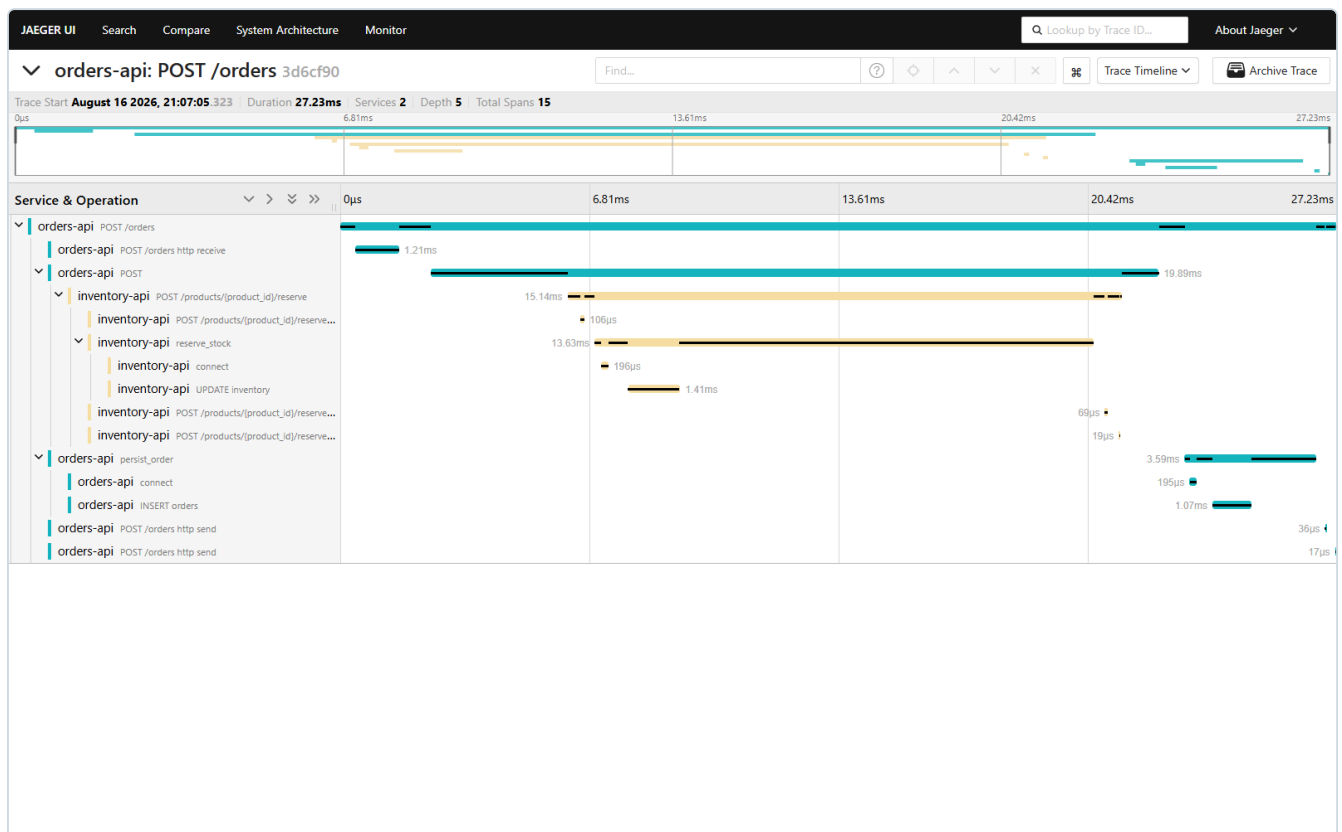


Figura 1 — Traza distribuida en Jaeger (local): spans de `orders-api` (`FastAPI`, `httpx`, `SQLAlchemy`, `persist_order`) y de `inventory-api` (`reserve_stock`), unificados por un único `trace_id` (propagación W3C).

La misma verificación se repitió en la nube: una traza distribuida en Cloud Trace abarca los spans de ambos servicios Cloud Run bajo un único `trace_id` (Figura 3, sección 8.3), y Cloud Logging permite filtrar los logs por ese mismo identificador.

6. Backends y visualización

6.1 OTEL Collector (local y nube)

Local: receiver OTLP (gRPC + HTTP) más `hostmetrics` y el self-scrape del collector; processors `memory_limiter` → `resource` → `resourcedetection` → `filter/health` (excluye los pings de `/health`) → `batch`; export **dual de trazas a Jaeger y Tempo** (Tempo además genera *service-graphs* y *span-metrics* hacia Prometheus por remote-write) y métricas a Prometheus (:8889). Nota: el exporter `jaeger` fue eliminado en collector-contrib ≥0.109, por lo que se usa `otlp` hacia el receptor OTLP de Jaeger.

Nube: un segundo Collector desplegado como servicio de Cloud Run recibe OTLP (gRPC sobre TLS/HTTP2) de los servicios y exporta con `googlecloud` (trazas → Cloud Trace, logs → Cloud Logging) y `googlemanagedprometheus` (métricas → Managed Prometheus).

6.2 Dashboards Grafana

El dashboard local evolucionó de la versión inicial de 6 paneles (Figura 2) a la versión actual de **10 paneles** con variables de plantilla `$service` y `$interval`, alineada con el laboratorio de referencia del curso:

Panel	Tipo	Consulta clave (PromQL)
SLI-1 Disponibilidad (SLO ≥ 99.5%)	gauge	<code>100 * sum(rate(http_requests_total{status=~"2.."}[\$interval])) / sum(rate(http_requests_total[\$interval]))</code>
Burn rate 1h (PAGE si > 14.4)	stat	<code>(rate 5xx 1h / rate total 1h) / 0.005</code>
Saturación (requests en vuelo)	stat	<code>sum by (service_name) (http_active_requests)</code>
SLI-2 Latencia p50/p95/p99 (SLO p95 < 500ms)	timeseries	<code>histogram_quantile(0.95, ... http_request_duration_milliseconds_bucket)</code>
SLI-3 Error rate por servicio + línea SLO 0.5%	timeseries	<code>100 * rate 5xx / rate total, by (service_name)</code>
SLI-4 Throughput (RPS)	timeseries	<code>sum by (service_name) (rate(http_requests_total[\$interval]))</code>
Latencia DB p99	timeseries	<code>histogram_quantile(0.99, ... db_operation_duration_milliseconds_bucket)</code>
CPU overhead (collector + host)	timeseries	<code>rate(otelcol_process_cpu_seconds) + system_cpu_time_seconds_total (hostmetrics)</code>
Collector: spans enviados/fallidos/cola	timeseries	<code>otelcol_exporter_sent_spans / send_failed / queue_size / queue_capacity</code>
Propagación W3C (cross-service)	tabla	<code>rate(service_b_calls_total) · rate(inventory_requests_total)</code>

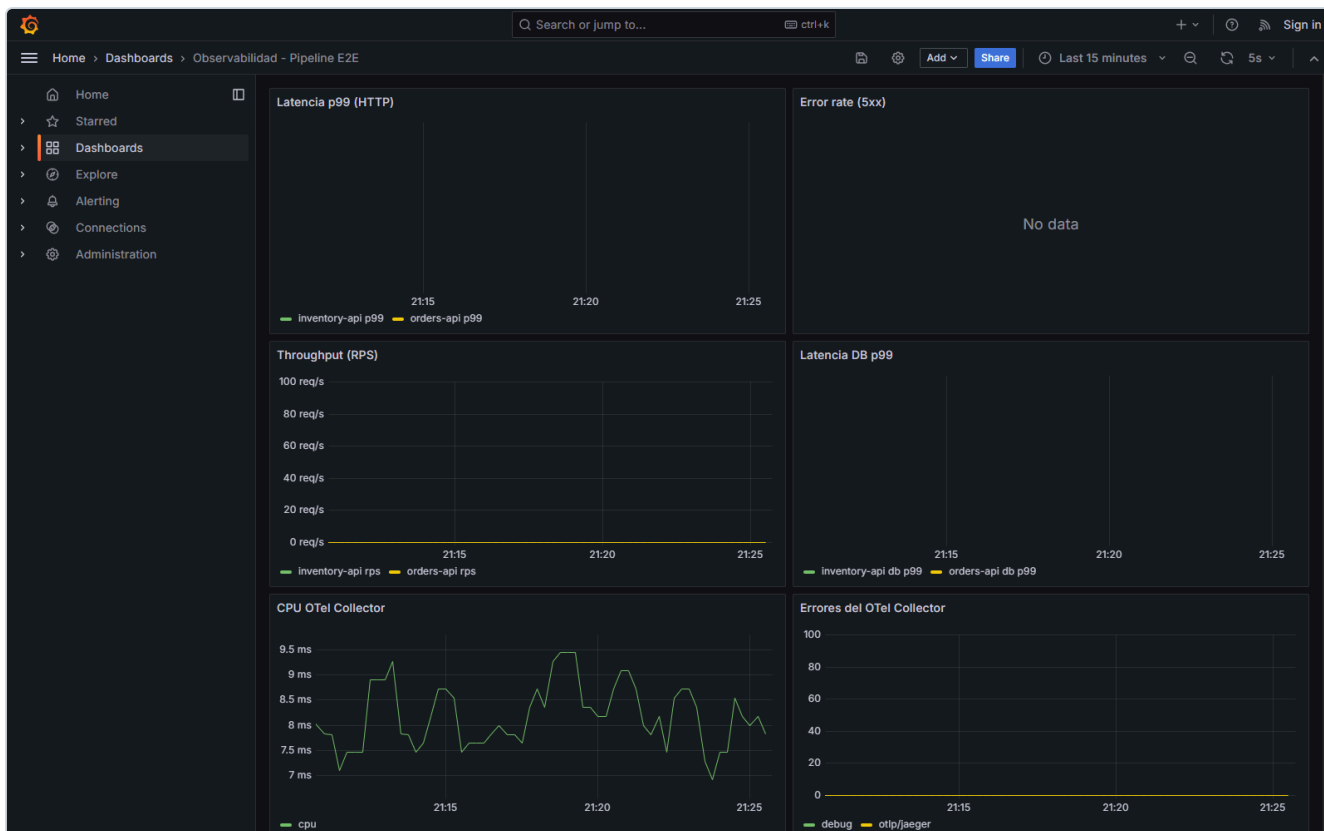


Figura 2 — Dashboard Grafana local (versión inicial de 6 paneles: 4 SLIs + salud del collector), posteriormente ampliado a los 10 paneles de la tabla anterior con SLOs, burn rate y variables de plantilla.

El **dashboard cloud** (Grafana en Cloud Run) replica 9 de estos paneles leyendo Managed Prometheus con PromQL a través del datasource de Google Cloud Monitoring (los paneles de CPU/collector no aplican porque el collector cloud no expone self-metrics hacia GMP).

7. Análisis de overhead (k6, local)

7.1 Metodología

- **Con instrumentación:** servicios bajo `opentelemetry-instrument` (auto-instrumentación + spans custom + métricas OTLP cada 5s).
- **Sin instrumentación:** los mismos servicios con uvicorn directo (override de compose); la telemetría queda en no-op.
- **Carga:** k6 con 50 VUs constantes sobre `POST /orders`, `product_id` rotado entre 100 productos para evitar contención de fila. (El script evolucionó después a escenarios warmup/sostenido/spike con `benchmark/analyze_overhead.py` para automatizar la comparación.)

7.2 Resultados (local, Docker Desktop Windows)

Métrica	Sin OTel	Con OTel	Overhead
Throughput	217 req/s	135 req/s	-38%
Latencia promedio	229 ms	368 ms	+61%
Latencia p95	525 ms	900 ms	+71%
Latencia p99	945 ms	1240 ms	+31%
Memoria orders-api	72 MiB	86 MiB	+19%
Memoria inventory-api	64 MiB	81 MiB	+27%
CPU (1 worker)	~97%	~98%	similar (ver interpretación)

7.3 Interpretación

El CPU se mantiene saturado en ambos casos (un único worker de uvicorn); el costo real se refleja en la caída de throughput a CPU comparable: **más CPU por request**. Contribuyen el *span fan-out* (~15–16 spans por orden, ~2000 spans/s a 135 req/s), el export de métricas cada 5s y la ausencia de muestreo (default siempre-muestrea). Con *tail-based sampling* (10–25%) el costo de export de trazas cae drásticamente — es la mitigación recomendada para producción.

8. Despliegue en GCP (Cloud Run + Cloud SQL)

8.1 Arquitectura del despliegue

El plan original contemplaba AWS ECS Fargate; se sustituyó por una arquitectura **serverless en GCP**, más simple de operar y cubierta por los créditos de la Free Trial. Los **cuatro** servicios corren en Cloud Run (región `us-central1`) y escalan a cero: `orders-api`, `inventory-api`, el **OTel Collector** (hub de observabilidad) y **Grafana**. La base de datos es Cloud SQL PostgreSQL 16 (`db-f1-micro`, conexión por socket Unix con `--add-cloudsql-instances`), que se apaga fuera de sesión (`activation-policy=NEVER`) como guardrail de coste.

8.2 CI/CD y seguridad (Workload Identity Federation)

- **Despliegue automático:** el merge a `main` dispara GitHub Actions con filtro de rutas (solo se redespiega el servicio cuyo código cambió); alternativas: `workflow_dispatch` manual y script local (`scripts/deploy.sh`).
- **Sin llaves:** la autenticación GitHub→GCP usa **Workload Identity Federation** con acceso restringido por condición al repositorio — no existen llaves de servicio almacenadas.
- **Secretos:** las cadenas de conexión (`DATABASE_URL`) viven en **Secret Manager** y Cloud Run las inyecta en runtime; nunca pasan por GitHub.
- **Imágenes pineadas por digest** (no `:latest`) para evitar revisiones cacheadas.

8.3 Telemetría nativa en GCP

El Collector exporta trazas a **Cloud Trace**, logs a **Cloud Logging** y métricas a **Managed Prometheus**; Grafana en Cloud Run consulta GMP con PromQL. La propagación W3C se verificó también en la nube:

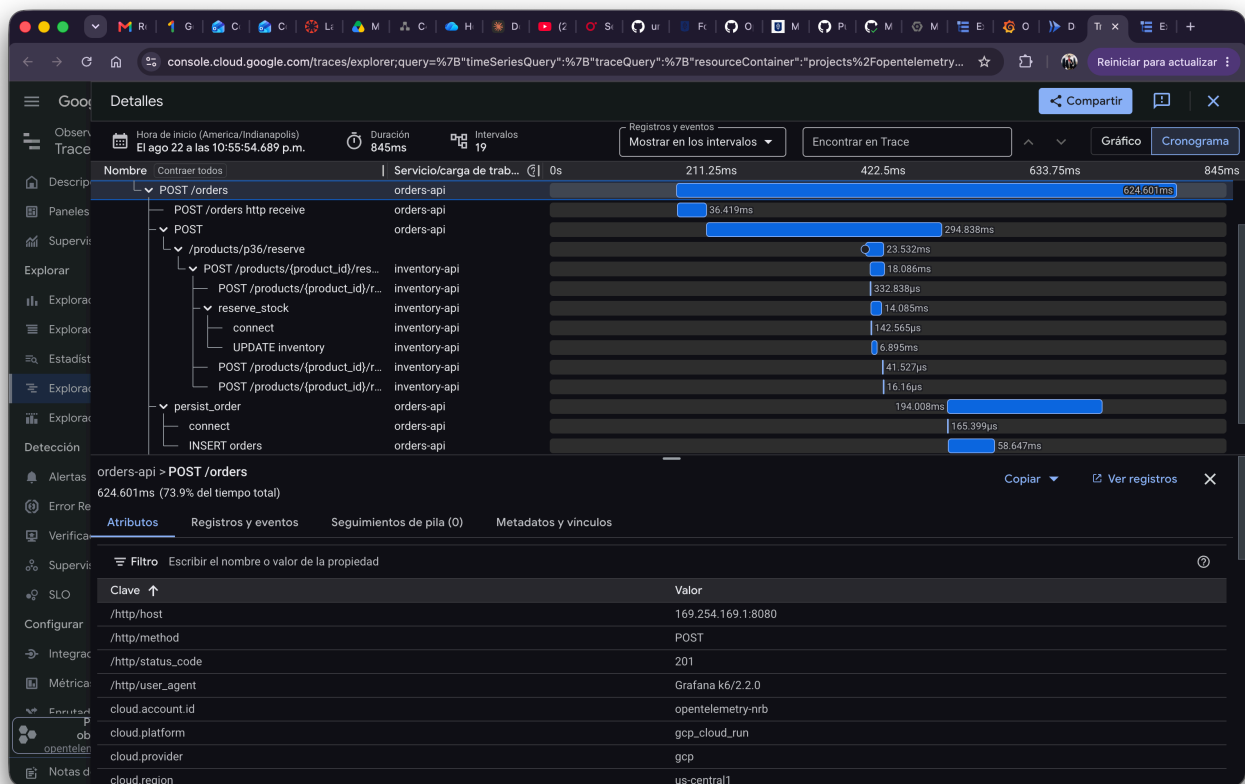


Figura 3 — Traza distribuida en Cloud Trace (845 ms, HTTP 201): el árbol abarca orders-api e inventory-api bajo un mismo trace_id, con los spans custom (reserve_stock, persist_order), las consultas SQL y atributos de recurso gcp_cloud_run.

9. Pruebas de carga en la nube y diagnóstico de incidente

Se ejecutaron pruebas k6 de **2 minutos sostenidos** contra el despliegue real. La primera produjo un incidente que fue diagnosticado íntegramente con la telemetría del propio pipeline:

Prueba	Éxito	RPS	p95	Hallazgo
50 VUs (pools 20/40)	14.4%	9	10 s	TooManyConnectionsError : los pools dimensionados para local agotaron las ~25 conexiones de db-f1-micro
15 VUs (pools 5/5)	100%	74	326 ms	Fix validado: pool configurable por entorno, cero errores, thresholds en verde
80 VUs (pools 5/5)	39%	142	1.38 s	El autoescalador multiplicó instancias (hasta 9) y con ellas las conexiones; instancias nuevas fallaron el arranque (conexión a DB en el startup)

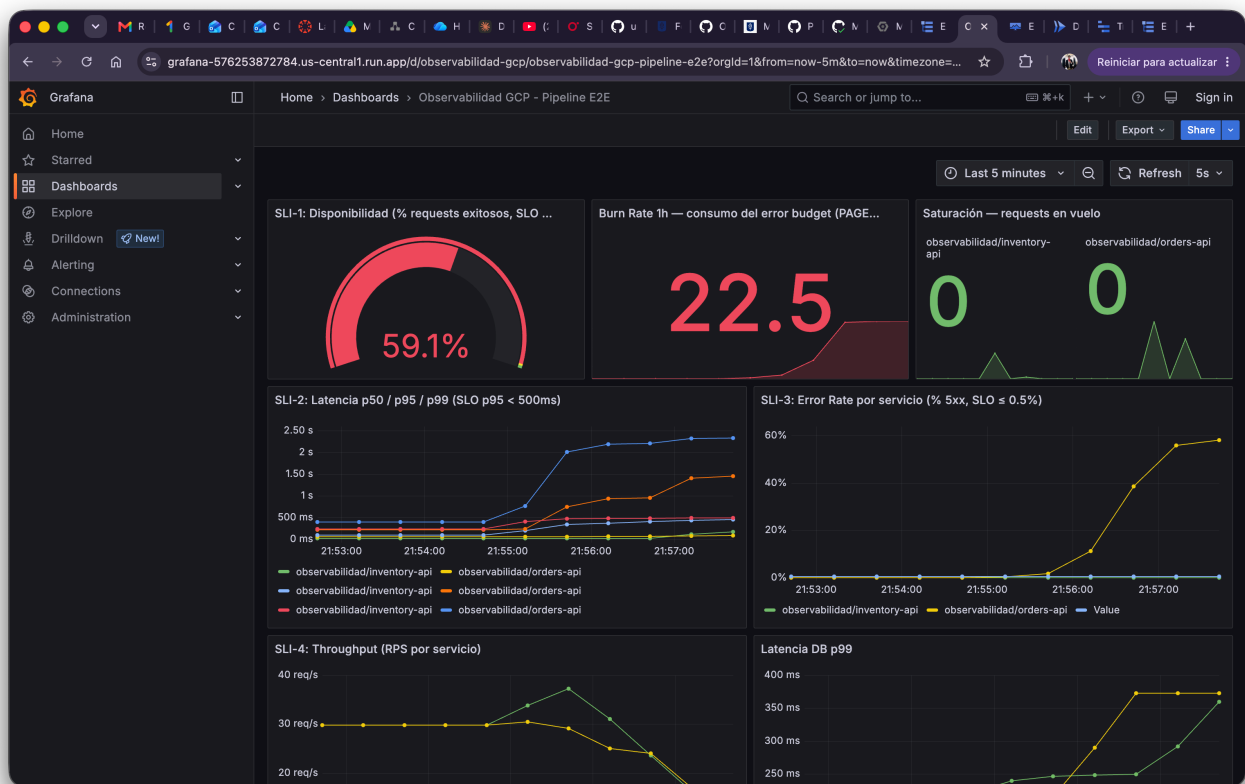


Figura 4 — Dashboard Grafana (Cloud Run) durante la prueba de 80 VUs: disponibilidad 59.1% (gauge en rojo), burn rate 22.5 (> 14.4 = alerta PAGE), percentiles p50/p95/p99 disparados sobre el SLO y error rate creciendo.

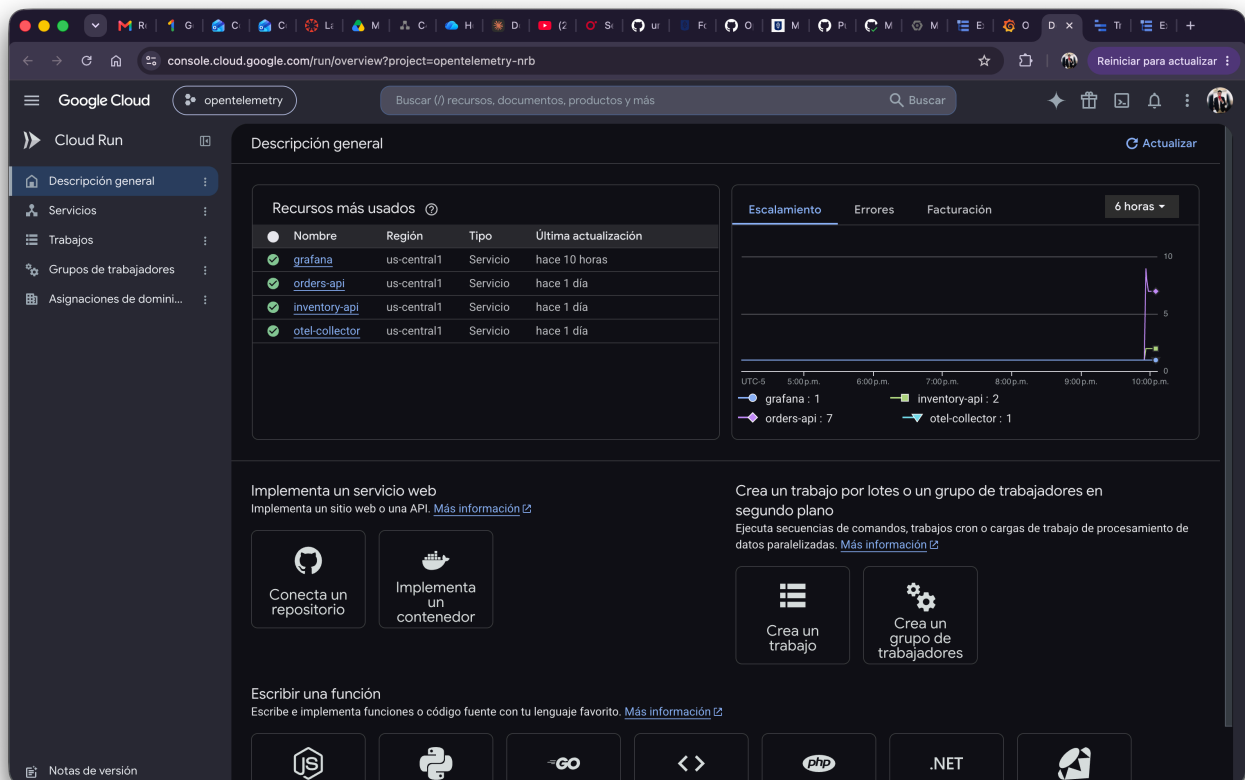


Figura 5 — Consola de Cloud Run: los cuatro servicios del pipeline y el gráfico de escalamiento donde `orders-api` salta de 1 a 7+ instancias durante la carga — el multiplicador de conexiones que agotó Cloud SQL.

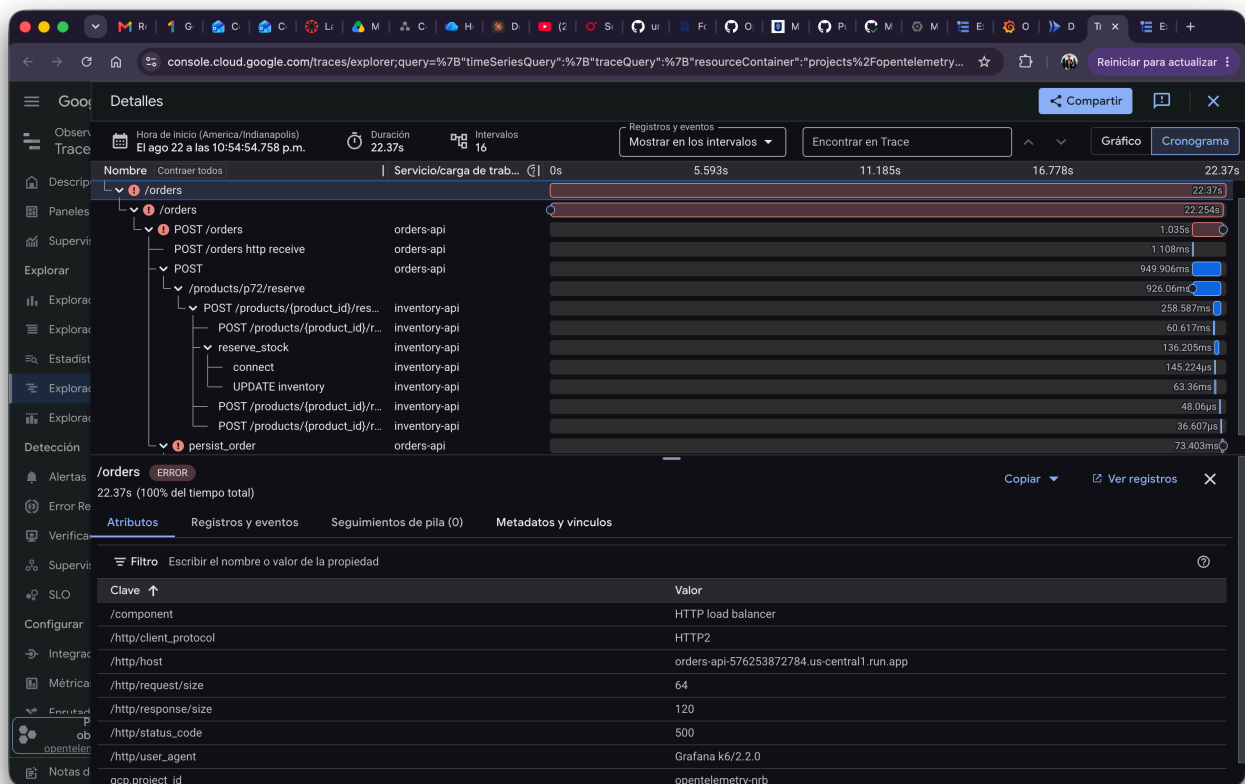


Figura 6 — Trazas del incidente en Cloud Trace: un POST /orders de 22.37 s terminando en HTTP 500; la cascada muestra la espera masiva antes del trabajo real (pool de conexiones agotado), evidencia con la que se diagnosticó la causa raíz.

Lecciones de la nube: el pool de conexiones debe dimensionarse por instancia contando el autoescalado (o limitarse con `max-instances`), y un arranque que se conecta a la base de datos es frágil bajo presión — la conexión perezosa con reintentos es la mitigación recomendada. Evidencias adicionales (trazas de error 503, logs correlacionados, evolución de los SLIs) en `screenshots_gcp/` del repositorio.

10. Guía de ejecución local y despliegue

10.1 Local (Docker Compose)

```
docker compose up --build -d
# Jaeger http://localhost:16686 · Tempo http://localhost:3200
# Prometheus http://localhost:9090 · Grafana http://localhost:3000
curl -X POST http://localhost:8080/orders -H "Content-Type: application/json" \
  -d '{"product_id":"p1","quantity":1,"customer_id":"demo"}'
```

10.2 Nube (GCP)

```
# encender la base de datos (los servicios se conectan a la DB al arrancar)
gcloud sql instances patch otel-pg --activation-policy=ALWAYS
# desplegar sin subir código (o merge a main para CI/CD)
./scripts/deploy.sh all
# apagar al terminar (guardrail de coste)
gcloud sql instances patch otel-pg --activation-policy=NEVER
```

Dashboard cloud: `grafana-576253872784.us-central1.run.app` · Infraestructura reproducible con `scripts/gcp_bootstrap.sh` · Guía completa en `DEPLOY.md`.

11. Decisiones de diseño y lecciones aprendidas

Problema encontrado	Solución aplicada
Exporter <code>jaeger</code> eliminado en collector-contrib 0.109	Exporter <code>otlp</code> → receptor OTLP de Jaeger
Imagen del collector distroless (sin shell/wget)	Sin healthcheck; el exporter OTLP reintenta solo
Métricas sin label <code>service_name</code>	<code>resource_to_telemetry_conversion: enabled</code>
SQLAlchemy async no emite <code>db.client.operation.duration</code>	Histograma custom <code>db_operation_duration</code>
Métricas con lag de 60s	<code>OTEL_METRIC_EXPORT_INTERVAL=5000</code>
Histograma de latencia con percentiles irreales	Registrar en ms : los buckets default de OTel (5, 10, 25...) asumen milisegundos
Contención de fila en el benchmark (todas las VUs sobre p1)	Repartir carga entre 100 productos — detectado vía trazas
Latencia elevada por conexión nueva por request	<code>httpx.AsyncClient</code> compartido (reutilización de conexiones)
Pools de DB dimensionados para local agotaban Cloud SQL (~25 conexiones)	Pool por entorno: <code>DB_POOL_SIZE / DB_MAX_OVERFLOW = 5/5</code> en Cloud Run
Credenciales de despliegue en CI	Workload Identity Federation (sin llaves) + Secret Manager
Cloud Run servía imagen cacheada bajo <code>:latest</code>	Despliegue pineado por digest de imagen
Plugin cloud-monitoring de Grafana pierde queries PromQL puras (bug en <code>migrateRequest</code>)	Workaround documentado: campo <code>timeSeriesList</code> dummy en cada target

12. Conclusiones

Se logró un pipeline de observabilidad completo y funcional con OpenTelemetry sobre microservicios FastAPI, cubriendo las tres señales y la correlación por `trace_id` exigida por el estándar W3C Trace Context, verificada tanto en local como en la nube. Las trazas distribuidas no solo cumplen el requisito de propagación: demostraron su valor operativo al aislar cuellos de botella durante el benchmark local (contención de fila, sobrecarga de conexiones) y al diagnosticar la causa raíz del incidente de saturación en la nube (agotamiento de conexiones amplificado por el autoescalado), guiando un fix que llevó la prueba de carga de 14% a **100% de éxito**.

El overhead de instrumentación es medible y aceptable a baja carga; bajo carga alta, el *span fan-out* y la ausencia de muestreo lo amplifican, siendo el *tail-based sampling* la mitigación recomendada para producción. El despliegue en **GCP Cloud Run + Cloud SQL**, con CI/CD sin llaves (Workload Identity Federation), secretos en Secret Manager, imágenes pineadas por digest y apagado de la base de datos fuera de sesión, mantiene el costo dentro de los créditos de la Free Trial sin sacrificar un pipeline de telemetría de grado productivo.